

SERVICIO NACIONAL DE APRENDIZAJE SENA

Centro de Gestión de Mercados, Logística y TIC



APP TRUEQUE MATCH

Desarrollo de Software a partir de la Integración de sus Módulos Componentes

Evidencia GA8-220501096-AA1-EV01

Estudiante: Gerson Jonnathan Lopez Oviedo

Ficha: 3186647

Instructor: Arnaldo Alfonso Montiel Brun

Programa: Tecnólogo en Análisis y Desarrollo de Software

Bogotá. Colombia 2026

Contenido

1. Introducción	4
2. Metodología de Desarrollo	5
2.1 Product Backlog (Historial de Requerimientos).....	5
2.2 Sprints de desarrollo	5
2.3 Roles	5
3. Requerimientos del Sistema	6
3.1 Requerimientos funcionales principales.....	6
3.2 Requerimientos no funcionales principales	6
4. IDE y Ambiente de Desarrollo	6
4.1 Herramientas del ambiente de desarrollo	8
5. Arquitectura y Capas de la Aplicación	8
5.1 Capa de presentación (Frontend)	8
5.2 Capa de lógica de negocio (Backend)	8
5.3 Capa de datos	8
6. Diagrama de Clases.....	9
7. Diagrama de Paquetes	10
8. Diagrama de Componentes	11
9. Mapa de Navegación.....	12
10. Mecanismos de Seguridad	13
10.1 Cifrado de contraseñas (bcrypt)	13
10.2 Mensajes de error genéricos.....	13
10.3 Sesiones de PHP	13
10.4 Prevención de inyección SQL.....	13
10.5 Validación de datos de entrada	13
10.6 CORS controlado	14
10.7 Mejora de seguridad pendiente.....	14
11. Codificación de los Módulos.....	14
11.1 Módulo de Autenticación.....	14
11.2 Módulo de Panel (Dashboard)	14
11.3 Módulo de APIs REST.....	14

11.4 Módulo de Aplicación Móvil	15
11.5 Evidencia del sistema en funcionamiento.....	16
12. Patrones de Diseño y Componentes Reutilizables	17
12.1 Separación de responsabilidades	17
12.2 Endpoint único por recurso (patrón REST)	18
12.3 Componente reutilizable: conexión centralizada	18
12.4 Función reutilizable: recálculo de reputación	18
12.5 Estructura de respuesta estandarizada	18
13. Librerías y Tecnologías por Capa.....	18
14. Pruebas Realizadas	19
14.1 Metodología de prueba	19
14.2 Evidencia de una prueba de refuerzo de seguridad	19
14.3 Casos de error controlados.....	19
14.4 Resumen de pruebas por módulo.....	20
15. Control de Versiones.....	20
16. Configuración de Servidores y Base de Datos	21
17. Ambientes de Desarrollo y Pruebas.....	23
18. Conclusión	24

1. Introducción

Mi nombre es Gerson Jonnathan López Oviedo, ficha 3186647, del programa Tecnólogo en Análisis y Desarrollo de Software. Este documento presenta la evidencia GA8-220501096-AA1-EV01, correspondiente al desarrollo de software a partir de la integración de los módulos que componen mi proyecto Trueque Match.

Trueque Match es una plataforma web y móvil que permite a los colombianos intercambiar productos, servicios, conocimientos y experiencias sin necesidad de dinero, promoviendo la economía colaborativa. A lo largo de este documento explico cómo codifiqué cada módulo del sistema, cómo se integran entre sí, qué arquitectura utilicé, qué mecanismos de seguridad implementé y cómo verifiqué que todo funcionara correctamente mediante pruebas reales.

Todo lo que se describe aquí corresponde a código que ya está escrito, subido a mi repositorio de control de versiones en GitHub y probado con datos reales, no a una propuesta teórica. Incluyo capturas de pantalla tomadas directamente de mi computador como evidencia de cada paso.

2. Metodología de Desarrollo

Para organizar el trabajo de este proyecto utilicé la metodología ágil Scrum. Elegí Scrum porque me permite dividir un proyecto grande, como es una plataforma completa de intercambios, en partes pequeñas y manejables que se pueden ir entregando y probando poco a poco, en vez de tratar de construir todo de una sola vez y descubrir los errores al final.

2.1 Product Backlog (Historial de Requerimientos)

El punto de partida de Scrum es el Product Backlog: una lista de todo lo que el sistema debe hacer, escrito desde el punto de vista del usuario. En mi caso, este backlog ya lo construí como parte de la evidencia GA7-220501096-AA4-EV03, donde definí 14 historias de usuario, 14 requerimientos funcionales y 14 requerimientos no funcionales. Cada historia de usuario describe una necesidad real, por ejemplo: "Como usuario registrado, quiero poder publicar una oferta de intercambio para que otras personas puedan verla y proponerme un trueque".

2.2 Sprints de desarrollo

Dividí el desarrollo en sprints cortos, cada uno enfocado en un módulo o grupo de módulos relacionados. Esto se refleja directamente en el historial de commits de mi repositorio: cada bloque de commits agrupados por fecha y por funcionalidad corresponde a un sprint de trabajo.

Sprint	Objetivo	Resultado (evidencia en commits)
1	Estructura base y conexión a base de datos	Estructura inicial HTML/CSS/JS + conexion.php
2	Autenticación de usuarios	registro.php con bcrypt, login.php con sesiones
3	Panel de usuario y gestión de ofertas	dashboard.php dinámico, CRUD de ofertas conectado
4	Capa de APIs REST	11 endpoints (login, ofertas, registro, solicitudes, trueques, evaluaciones, favoritos, notificaciones, reportes)
5	Pruebas y aseguramiento de calidad	Pruebas en Postman, corrección de duplicados, refuerzo de seguridad

2.3 Roles

En este proyecto asumí los roles de Product Owner (definiendo qué necesita el sistema, con ayuda de las historias de usuario asignadas por el instructor) y de Development Team (codificando cada módulo). El instructor Arnaldo Alfonso Montiel Brun actuó como validador de cada entrega, revisando las evidencias correspondientes a cada sprint.

3. Requerimientos del Sistema

Los requerimientos completos, con sus 14 historias de usuario, 14 requerimientos funcionales y 14 requerimientos no funcionales, ya quedaron documentados en la evidencia GA7-220501096-AA4-EV03. Aquí resumo los más relevantes para entender qué módulos había que construir:

3.1 Requerimientos funcionales principales

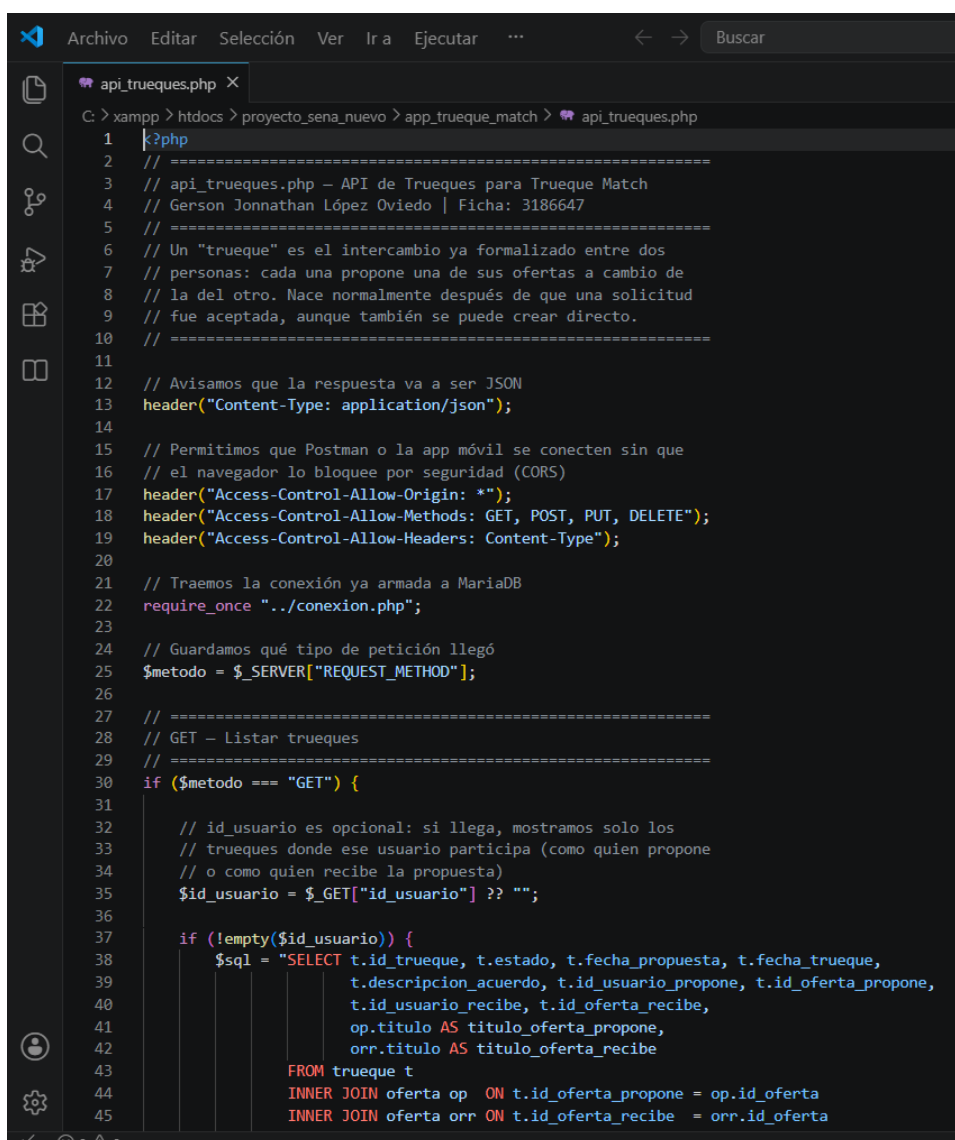
- El sistema debe permitir el registro de nuevos usuarios con validación de datos.
- El sistema debe permitir iniciar sesión de forma segura.
- El sistema debe permitir crear, editar, ver y eliminar ofertas de intercambio.
- El sistema debe permitir solicitar contacto sobre una oferta antes de formalizar un trueque.
- El sistema debe permitir proponer, aceptar y completar trueques.
- El sistema debe permitir calificar a la otra persona después de un trueque.
- El sistema debe permitir guardar ofertas como favoritas.
- El sistema debe notificar al usuario sobre eventos relevantes.
- El sistema debe permitir reportar comportamientos indebidos.

3.2 Requerimientos no funcionales principales

- Las contraseñas deben almacenarse cifradas, nunca en texto plano.
- El sistema debe responder en formato JSON para permitir su consumo desde la app móvil.
- El sistema debe funcionar tanto en navegador web como en dispositivo móvil Android.
- El código debe estar versionado en un repositorio Git accesible.
- El sistema debe validar los datos de entrada antes de guardarlos en la base de datos.

4. IDE y Ambiente de Desarrollo

Para codificar todos los módulos de Trueque Match utilicé Visual Studio Code como entorno de desarrollo (IDE), por ser una herramienta gratuita, ligera y con buen soporte para PHP, HTML, CSS y JavaScript mediante extensiones. A continuación muestro el explorador de archivos de VS Code con la estructura real de mi proyecto, y un ejemplo de código fuente abierto y comentado.



```

1  <?php
2  // =====
3  // api_trueques.php - API de Trueques para Trueque Match
4  // Gerson Jonnathan López Oviedo | Ficha: 3186647
5  // =====
6  // Un "trueque" es el intercambio ya formalizado entre dos
7  // personas: cada una propone una de sus ofertas a cambio de
8  // la del otro. Nace normalmente después de que una solicitud
9  // fue aceptada, aunque también se puede crear directo.
10 // =====
11
12 // Avisamos que la respuesta va a ser JSON
13 header("Content-Type: application/json");
14
15 // Permitimos que Postman o la app móvil se conecten sin que
16 // el navegador lo bloquee por seguridad (CORS)
17 header("Access-Control-Allow-Origin: *");
18 header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE");
19 header("Access-Control-Allow-Headers: Content-Type");
20
21 // Traemos la conexión ya armada a MariaDB
22 require_once "../conexion.php";
23
24 // Guardamos qué tipo de petición llegó
25 $metodo = $_SERVER["REQUEST_METHOD"];
26
27 // =====
28 // GET - Listar trueques
29 // =====
30 if ($metodo === "GET") {
31     // id_usuario es opcional: si llega, mostramos solo los
32     // trueques donde ese usuario participa (como quien propone
33     // o como quien recibe la propuesta)
34     $id_usuario = $_GET["id_usuario"] ?? "";
35
36     if (!empty($id_usuario)) {
37         $sql = "SELECT t.id_trueque, t.estado, t.fecha_propuesta, t.fecha_trueque,
38                 t.descripcion_acuerdo, t.id_usuario_propone, t.id_oferta_propone,
39                 t.id_usuario_recibe, t.id_oferta_recibe,
40                 op.titulo AS titulo_oferta_propone,
41                 orr.titulo AS titulo_oferta_recibe
42                 FROM trueque t
43                 INNER JOIN oferta op ON t.id_oferta_propone = op.id_oferta
44                 INNER JOIN oferta orr ON t.id_oferta_recibe = orr.id_oferta

```

Figura 1. Visual Studio Code con el archivo `api_trueques.php` abierto, mostrando el código comentado línea por línea.

Como se observa en la figura, cada archivo de mi proyecto incluye comentarios explicando qué hace cada bloque de código. Esto no solo es una buena práctica de programación, sino que también facilita que otra persona (o yo mismo, meses después) entienda la lógica sin tener que descifrarla desde cero.

4.1 Herramientas del ambiente de desarrollo

Herramienta	Uso en el proyecto
Visual Studio Code	Edición y codificación de todos los archivos PHP, HTML, CSS y JavaScript
XAMPP	Servidor local: Apache (servidor web) + MariaDB (base de datos) para desarrollo y pruebas
phpMyAdmin	Administración visual de la base de datos trueque_match_db
Postman	Pruebas de las APIs REST, simulando peticiones reales sin necesidad de interfaz gráfica
Git Bash	Control de versiones: commits y sincronización con GitHub
GitHub	Repositorio remoto público donde vive la versión oficial del código
Expo (React Native)	Desarrollo y prueba de la aplicación móvil en Android

5. Arquitectura y Capas de la Aplicación

Trueque Match está organizado en tres capas claramente separadas, siguiendo el modelo cliente-servidor con acceso a base de datos. Separar el proyecto en capas permite que cada parte se pueda modificar de forma independiente: por ejemplo, puedo cambiar el diseño visual sin tocar la lógica de negocio, o cambiar una consulta SQL sin afectar cómo se ve la página.

5.1 Capa de presentación (Frontend)

Es todo lo que el usuario ve e interactúa directamente: las páginas HTML, los estilos CSS que le dan la identidad visual a Trueque Match (colores rojo y negro, tipografía Bebas Neue y Nunito), y el JavaScript que maneja la interactividad, como abrir el formulario de nueva oferta o mostrar mensajes.

5.2 Capa de lógica de negocio (Backend)

Es el conjunto de archivos PHP que procesan la información: verifican que una contraseña sea correcta, calculan la reputación de un usuario, deciden si una solicitud se puede aceptar, entre otras reglas propias del negocio de Trueque Match. Esta capa se divide en dos partes: las páginas PHP tradicionales (login, registro, dashboard) que generan HTML, y las APIs REST que generan JSON para ser consumidas por Postman o por la aplicación móvil.

5.3 Capa de datos

Es la base de datos MariaDB, con sus 10 tablas normalizadas hasta tercera forma normal. Ningún módulo accede a esta capa directamente sin pasar por el archivo conexion.php, que centraliza la conexión y evita que cada archivo tenga que repetir esa lógica.

6. Diagrama de Clases

El siguiente diagrama representa las entidades (clases) del sistema, sus atributos y las relaciones entre ellas. Lo construí en MySQL Workbench a partir del modelo relacional ya normalizado de la base de datos, lo cual es una práctica común en proyectos donde la lógica de negocio está fuertemente ligada a la estructura de datos, como es el caso de Trueque Match.

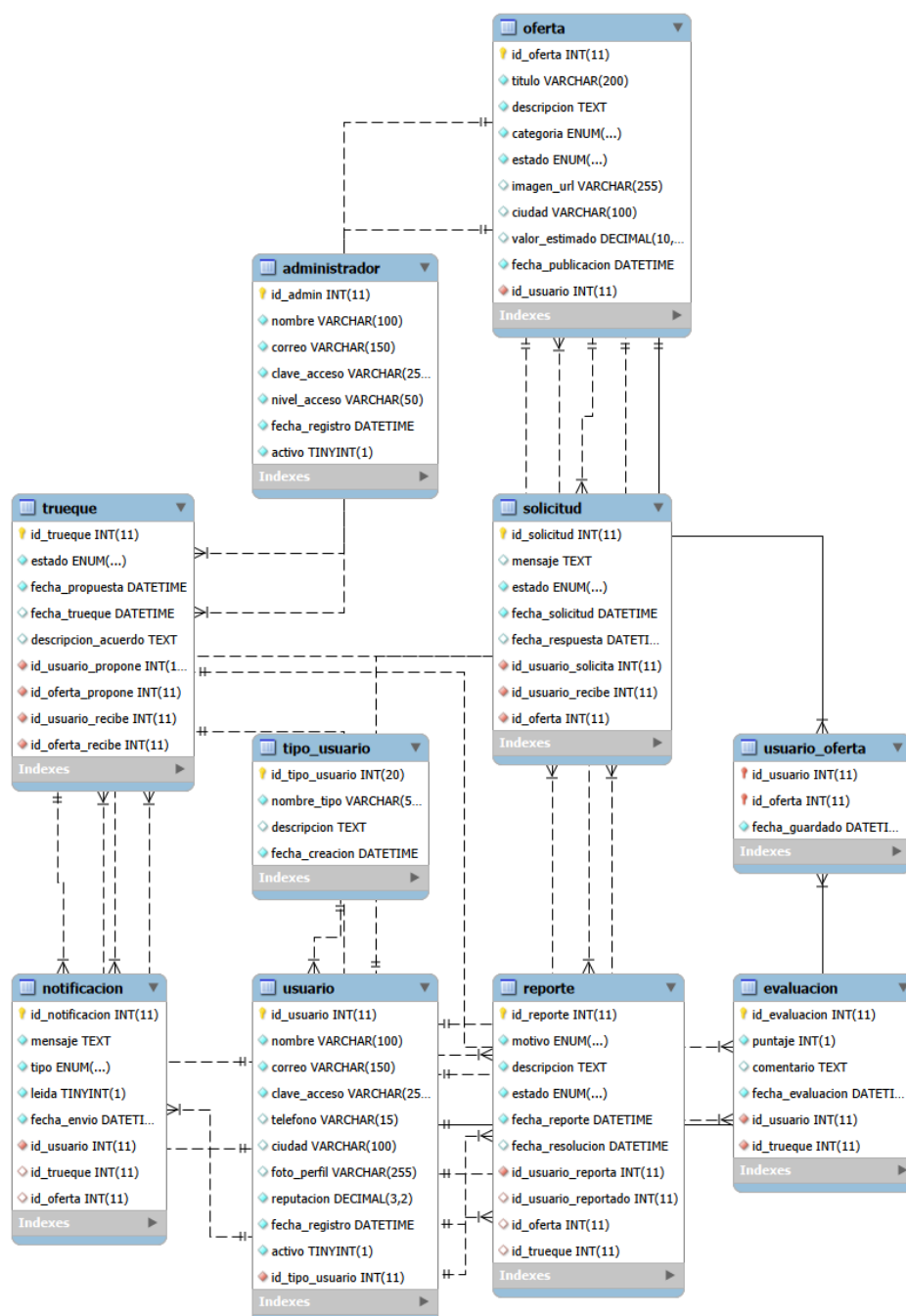


Figura 2. Diagrama de clases (modelo EER) de Trueque Match con las 10 entidades del sistema.

Cada rectángulo del diagrama representa una clase/entidad con sus atributos: usuario, oferta, trueque, solicitud, evaluación, notificación, reporte, administrador, tipo_usuario y usuario_oferta (esta última es una clase asociativa que resuelve la relación de muchos a muchos entre usuarios y sus ofertas favoritas). Las líneas entre las clases representan las relaciones y su cardinalidad: por ejemplo, un usuario puede tener muchas ofertas, pero cada oferta pertenece a un único usuario.

7. Diagrama de Paquetes

Un paquete agrupa componentes relacionados entre sí, de forma parecida a como una carpeta agrupa archivos relacionados en un computador. Organizar el código en paquetes ayuda a entender rápidamente qué hace cada parte del sistema sin tener que leer archivo por archivo.

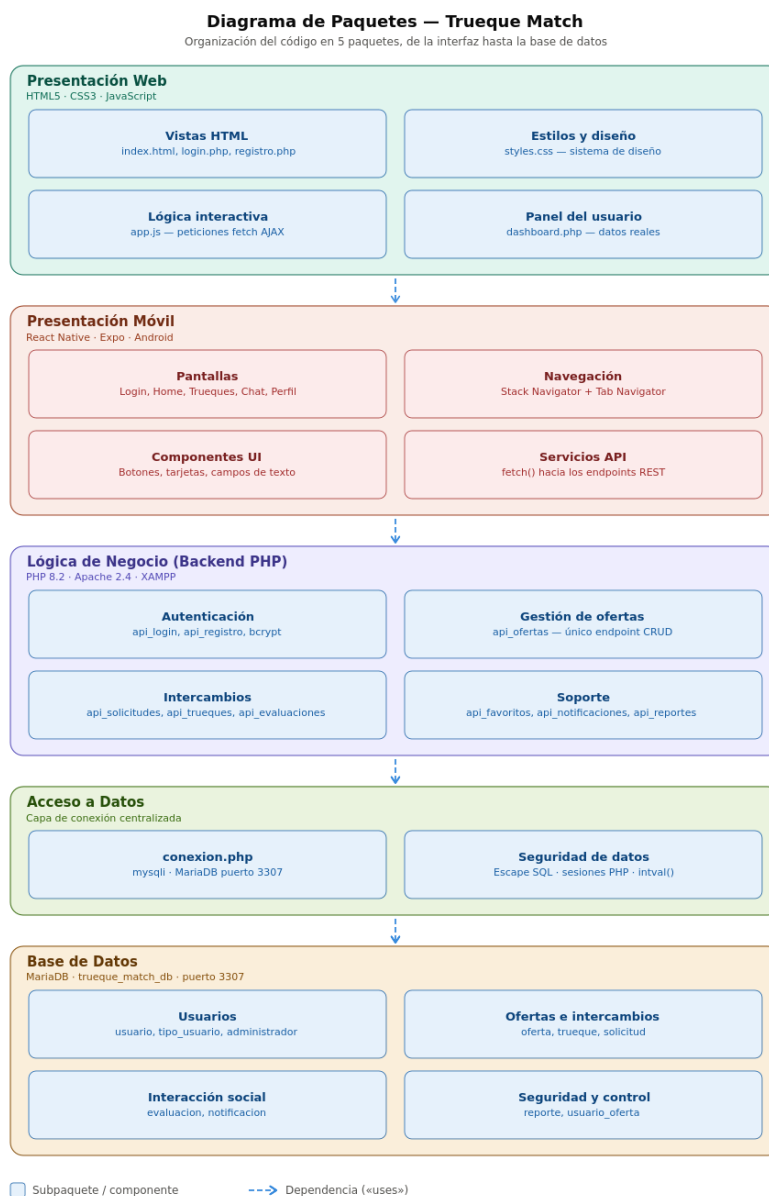


Figura 3. Diagrama de paquetes de Trueque Match.

El proyecto se organiza en cinco paquetes principales: Frontend Web (interfaz visual), Backend PHP (que a su vez contiene el módulo de autenticación, el módulo de panel y el módulo de APIs REST), Acceso a Datos (la conexión centralizada a MariaDB), y App Móvil (el cliente en React Native). Las flechas indican dependencias: por ejemplo, el paquete de APIs REST depende del paquete de Acceso a Datos, porque necesita consultar y modificar información en la base de datos.

Dentro del paquete de APIs REST, a su vez, agrupé los 11 endpoints en cuatro sub-paquetes lógicos según su responsabilidad: Usuarios (login y registro), Ofertas (el catálogo de publicaciones), Intercambio (solicitudes, trueques y evaluaciones, que son el corazón funcional del negocio) y Soporte (favoritos, notificaciones y reportes, que complementan la experiencia pero no son el flujo principal).

8. Diagrama de Componentes

El diagrama de componentes muestra las piezas de software que conforman el sistema y cómo se comunican entre sí a través de sus interfaces, sin importar los detalles internos de cada una. Es útil para entender el flujo completo de una petición, desde que sale del cliente hasta que llega a la base de datos y regresa.

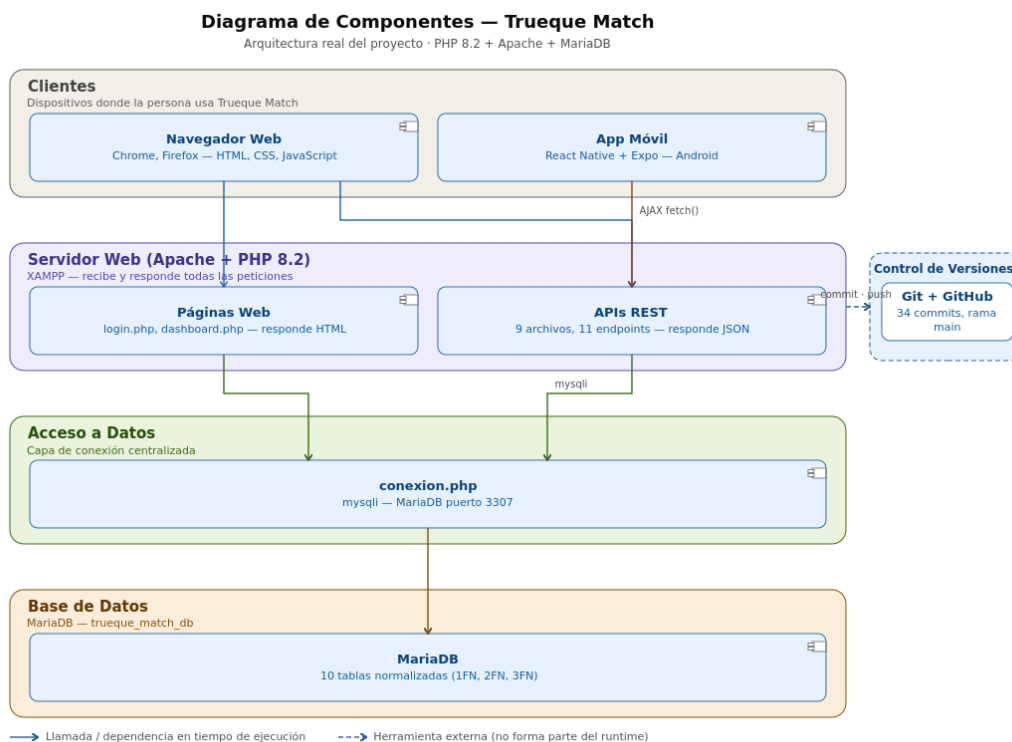


Figura 4. Diagrama de componentes de Trueque Match.

Tres tipos de clientes pueden comunicarse con el sistema: el navegador web (usado por una persona normal), la aplicación móvil en React Native (que consume las APIs directamente en formato JSON) y Postman (usado durante las pruebas técnicas). Los tres se conectan al servidor Apache por el puerto 80 mediante peticiones HTTP. Apache enruta cada petición según la URL: si es una página como login.php la envía al componente de Páginas PHP, y si es un archivo api_*.php la envía al componente de APIs REST. Ambos componentes, al necesitar datos, se conectan a MariaDB en el puerto 3307 usando la extensión mysqli de PHP. phpMyAdmin es una herramienta aparte que se conecta directamente a la base de datos para tareas de administración.

9. Mapa de Navegación

El mapa de navegación representa el recorrido que puede hacer un usuario a través de las distintas pantallas de la aplicación web, desde que llega por primera vez hasta que cierra sesión.

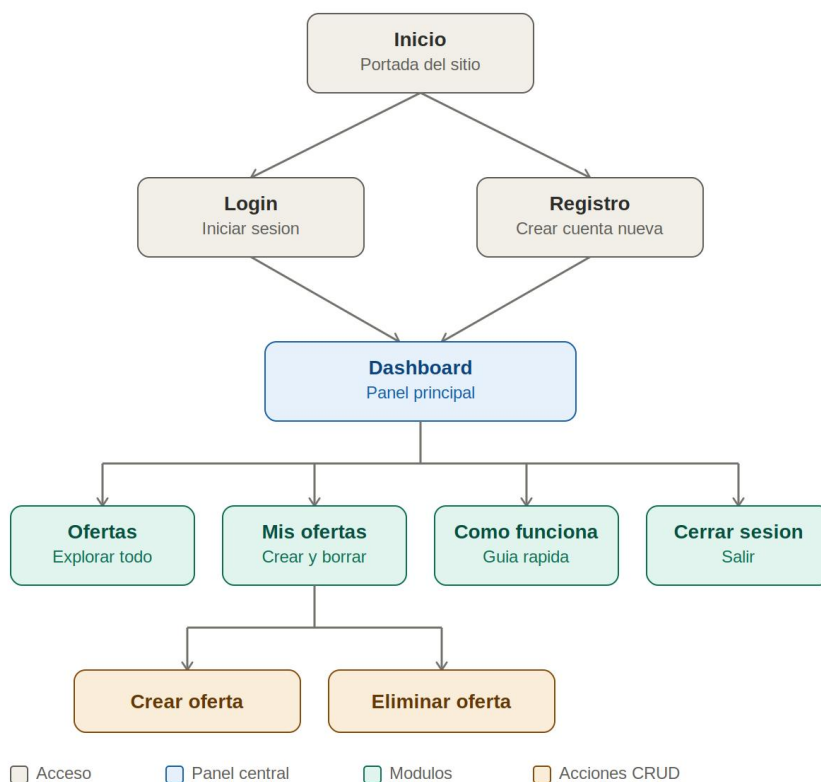


Figura 5. Mapa de navegación de Trueque Match.

Un usuario nuevo llega a la página principal (index.html), desde donde puede registrarse o iniciar sesión. Una vez dentro, el dashboard.php funciona como centro de control desde el cual se puede navegar a explorar ofertas, ver mis ofertas, publicar una nueva oferta mediante un formulario modal, revisar mis trueques, entrar al chat, ver notificaciones, editar mi perfil o cerrar sesión. Esta

estructura centralizada facilita que el usuario nunca se sienta perdido, porque siempre puede volver al panel principal con un clic.

10. Mecanismos de Seguridad

La seguridad de la información de los usuarios fue una prioridad desde el diseño de la base de datos hasta la codificación de cada módulo. A continuación explico los mecanismos que implementé, con el fragmento de código real donde se aplican.

10.1 Cifrado de contraseñas (bcrypt)

Ninguna contraseña se guarda en texto plano. Uso la función `password_hash()` de PHP con el algoritmo `bcrypt`, que convierte la contraseña en un texto cifrado imposible de revertir. Al iniciar sesión, se usa `password_verify()` para comparar la contraseña ingresada contra ese texto cifrado, sin necesidad de descifrarlo nunca.

10.2 Mensajes de error genéricos

Cuando alguien falla al iniciar sesión, el sistema responde siempre con el mismo mensaje ("credenciales incorrectas"), sin especificar si fue el correo o la contraseña lo que falló. Esto evita que una persona con malas intenciones pueda usar los mensajes de error para descubrir qué correos sí están registrados en el sistema.

10.3 Sesiones de PHP

Uso `session_start()` y las variables `$_SESSION` para mantener identificado a un usuario mientras navega, y las páginas protegidas como el dashboard verifican con `isset($_SESSION['usuario_id'])` que exista una sesión activa antes de mostrar información privada.

10.4 Prevención de inyección SQL

En las funciones de creación y edición de usuarios y ofertas, los datos que llegan del usuario se limpian con `mysqli_real_escape_string()` antes de insertarlos en una consulta SQL, evitando que alguien intente inyectar código malicioso a través de un formulario.

10.5 Validación de datos de entrada

Cada API valida los datos antes de guardarlos: el correo se valida con `filter_var()` y `FILTER_VALIDATE_EMAIL`, los puntajes de evaluación se validan que estén entre 1 y 5, las categorías y estados se validan contra una lista fija de valores permitidos (por ejemplo, una oferta solo

puede tener categoría producto, servicio, conocimiento o experiencia), y los identificadores numéricos se validan con `intval()` para evitar que lleguen datos con formato incorrecto.

10.6 CORS controlado

Cada API incluye las cabeceras `Access-Control-Allow-Origin` y `Access-Control-Allow-Methods`, que definen explícitamente qué métodos HTTP están permitidos (GET, POST, PUT, DELETE), evitando respuestas a métodos no contemplados.

10.7 Mejora de seguridad pendiente

Como parte de la revisión honesta de mi propio código, identifiqué que algunas versiones anteriores de mis APIs llamaban a `session_start()` con un comentario que decía "verificamos que el usuario esté logueado", pero en realidad nunca se llegaba a revisar el contenido de `$_SESSION`. Esta es una mejora de seguridad que dejo identificada como trabajo futuro: agregar una validación real de sesión activa en cada API antes de permitir operaciones de escritura.

11. Codificación de los Módulos

A continuación explico cómo codifiqué cada módulo del sistema, agrupados según su responsabilidad, siguiendo la misma organización del diagrama de paquetes presentado anteriormente.

11.1 Módulo de Autenticación

Compuesto por `registro.php`, `login.php` y `cerrar_sesion.php`. Se encarga de crear cuentas nuevas con contraseña cifrada, verificar credenciales al iniciar sesión y destruir la sesión al salir con `session_destroy()`.

11.2 Módulo de Panel (Dashboard)

Compuesto por `dashboard.php` y `mis_ofertas_ajax.php`. Muestra en tiempo real la información del usuario que inició sesión: cuántas ofertas tiene, cuántos trueques, su reputación y sus favoritos, todo obtenido con consultas SQL reales, no con datos de ejemplo fijos en el código.

11.3 Módulo de APIs REST

Es el módulo más grande del proyecto: 11 archivos independientes, cada uno responsable de una tabla de la base de datos, todos siguiendo el mismo patrón de codificación para mantener consistencia. La siguiente tabla resume cada uno:

Archivo	Métodos HTTP	Responsabilidad
api_login.php	POST	Verifica credenciales y devuelve los datos del usuario
api_registro.php	POST	Crea una cuenta de usuario nueva con validaciones completas
api_ofertas.php	GET, POST, PUT, DELETE	Endpoint único y oficial para listar, crear, editar y eliminar ofertas
api_solicitudes.php	GET, POST, PUT, DELETE	Gestiona el contacto previo entre dos usuarios antes de un trueque
api_trueques.php	GET, POST, PUT, DELETE	Gestiona la propuesta, aceptación y finalización de un intercambio; al completarse, actualiza automáticamente el estado de las ofertas involucradas
api_evaluaciones.php	GET, POST, PUT, DELETE	Gestiona las calificaciones entre usuarios; recalcula automáticamente la reputación del usuario evaluado
api_favoritos.php	GET, POST, DELETE	Gestiona la lista de ofertas guardadas por cada usuario
api_notificaciones.php	GET, POST, PUT, DELETE	Gestiona las alertas del sistema para cada usuario
api_reportes.php	GET, POST, PUT, DELETE	Gestiona las denuncias sobre usuarios, ofertas o trueques

Todas las APIs siguen la misma estructura interna: primero definen las cabeceras de tipo de contenido y CORS, luego identifican el método HTTP recibido con `$_SERVER['REQUEST_METHOD']`, y según ese método ejecutan un bloque distinto (GET para consultar, POST para crear, PUT para editar, DELETE para eliminar), cada uno con sus propias validaciones antes de tocar la base de datos.

11.4 Módulo de Aplicación Móvil

Desarrollado en React Native con Expo, cuenta con 6 pantallas (Login, Registro, Home, Trueques, Chat y Perfil) que consumen las mismas APIs REST del backend, garantizando que la información esté siempre sincronizada entre la web y el celular.

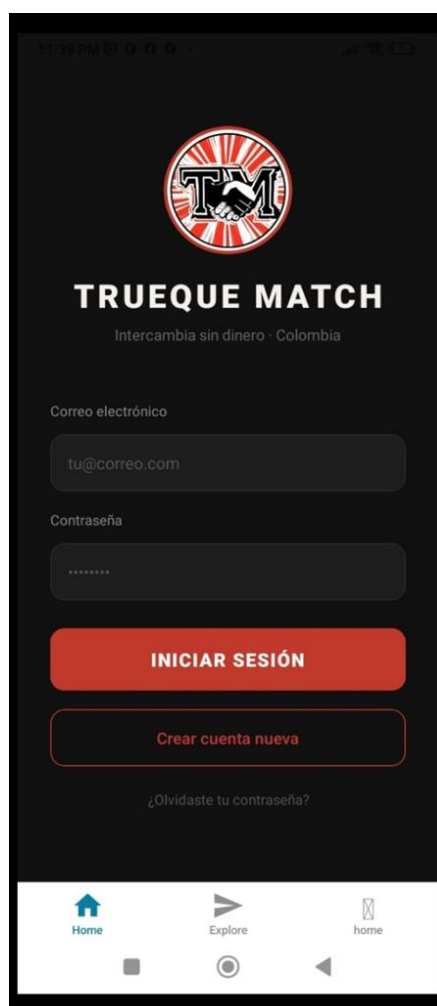


Figura 6. Pantalla de inicio de sesión de la aplicación móvil Trueque Match, funcionando en un dispositivo Android real.

11.5 Evidencia del sistema en funcionamiento

Para demostrar que los módulos están efectivamente integrados y funcionando juntos, incluyo capturas del sistema completo en ejecución, desde el inicio de sesión hasta la gestión de ofertas.

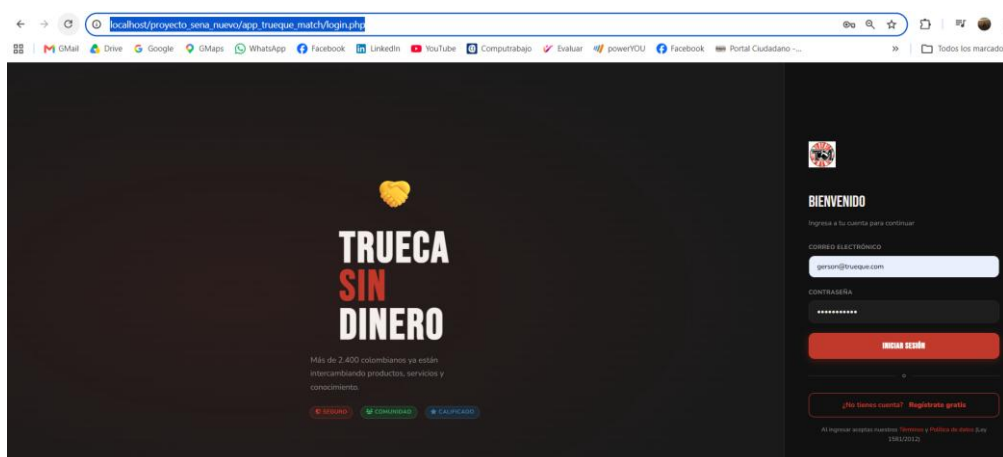


Figura 7. Pantalla de login del sistema web, funcionando en el navegador.

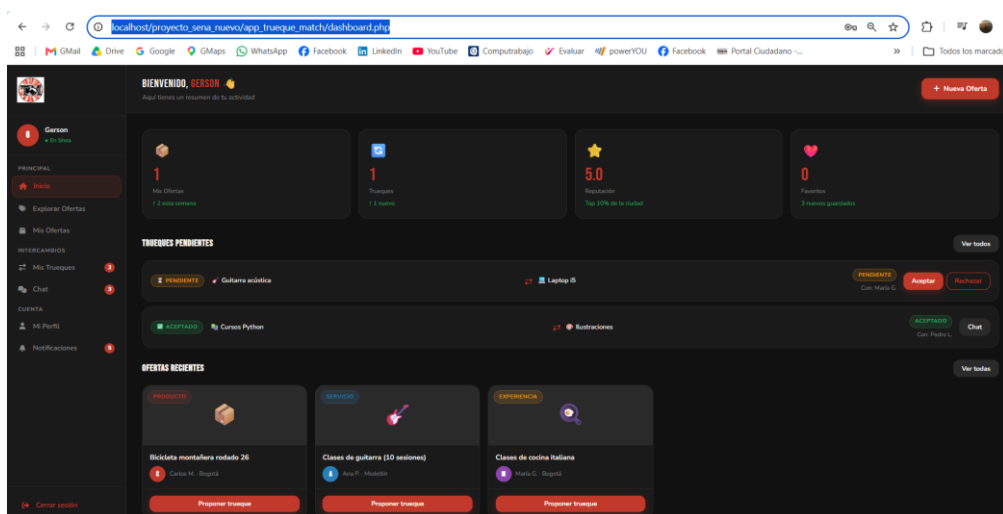


Figura 8. Dashboard mostrando datos reales del usuario tras iniciar sesión correctamente.

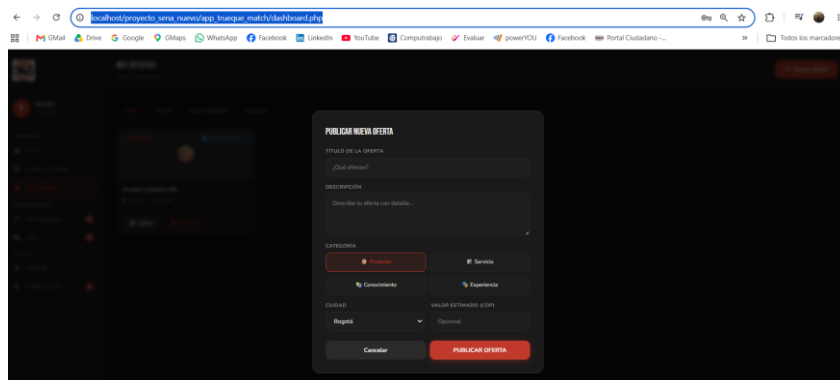


Figura 9. Formulario modal para publicar una nueva oferta, integrado dentro del dashboard.

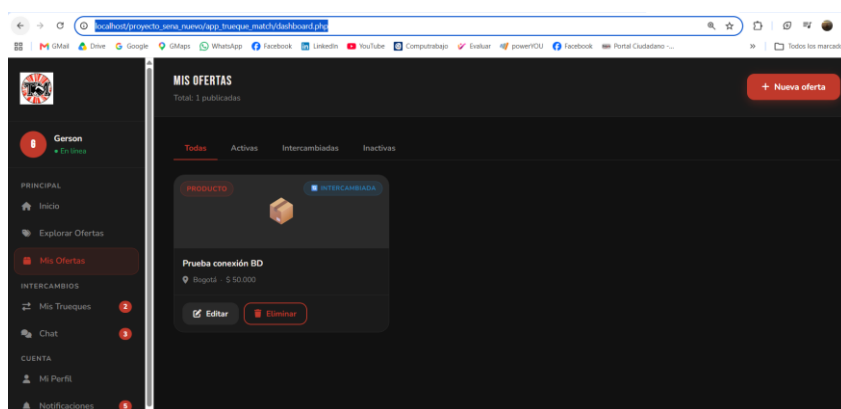


Figura 10. Sección "Mis Ofertas", mostrando el estado real de una oferta ya intercambiada.

12. Patrones de Diseño y Componentes Reutilizables

12.1 Separación de responsabilidades

Aunque no utilicé un framework que imponga formalmente el patrón Modelo-Vista-Controlador, apliqué su principio central: separar el acceso a datos (`conexion.php`), la lógica de negocio (cada archivo PHP y cada API) y la presentación (los archivos HTML y CSS), evitando mezclar consultas SQL dentro del HTML.

12.2 Endpoint único por recurso (patrón REST)

Cada API sigue el principio REST de que un mismo recurso (por ejemplo, una oferta) debe tener un único endpoint que responda distinto según el verbo HTTP utilizado, en vez de crear un archivo separado por cada acción. Este patrón lo apliqué de forma consistente en `api_ofertas.php`, `api_solicitudes.php`, `api_trueques.php`, `api_evaluaciones.php`, `api_favoritos.php`, `api_notificaciones.php` y `api_reportes.php`.

12.3 Componente reutilizable: conexión centralizada

El archivo `conexion.php` es el ejemplo más claro de componente reutilizable en el proyecto: en vez de que cada uno de los más de 20 archivos PHP abra su propia conexión a la base de datos, todos incluyen este único archivo con `require_once`, reutilizando la misma lógica de conexión.

12.4 Función reutilizable: recálculo de reputación

Dentro de `api_evaluaciones.php` escribí la función `recalcular_reputacion()`, que se reutiliza en tres puntos distintos del mismo archivo (al crear, editar y eliminar una evaluación), evitando repetir el mismo bloque de código SQL tres veces.

12.5 Estructura de respuesta estandarizada

Todas las APIs devuelven sus respuestas en un formato JSON consistente, con las claves `status`, `mensaje` y `data` (o el nombre específico del identificador creado/editado/eliminado). Esta estandarización facilita que cualquier cliente, ya sea Postman, la app móvil o una futura interfaz, sepa siempre qué esperar de cualquier endpoint del sistema.

13. Librerías y Tecnologías por Capa

Capa	Tecnologías y librerías
Presentación (Web)	HTML5, CSS3, JavaScript nativo (sin frameworks de frontend)
Presentación (Móvil)	React Native, Expo (framework de desarrollo móvil multiplataforma)
Lógica de negocio	PHP 8.2 nativo, extensión <code>mysqli</code> para la conexión a base de datos, funciones nativas <code>password_hash()</code> y <code>password_verify()</code> para seguridad

Datos	MariaDB (compatible con MySQL), motor de almacenamiento InnoDB con soporte de llaves foráneas
Herramientas de soporte	XAMPP (Apache + MariaDB + PHP integrados), phpMyAdmin, Postman, Git

14. Pruebas Realizadas

No basta con escribir el código: hay que comprobar que funciona con datos reales. Para esto utilicé Postman, una herramienta que permite enviarle peticiones directas a cada API sin pasar por la interfaz gráfica, y así confirmar que el backend responde correctamente de forma aislada.

14.1 Metodología de prueba

Para cada uno de los 11 endpoints probé sus operaciones principales (crear, leer, editar, eliminar según aplique) con datos reales de mi base de datos, verificando además en phpMyAdmin que los cambios efectivamente quedaran guardados, y no solo que la respuesta JSON se viera correcta en pantalla.

14.2 Evidencia de una prueba de refuerzo de seguridad

La siguiente captura corresponde a una prueba de DELETE sobre `api_ofertas.php`, posterior al refuerzo de seguridad que le agregué al endpoint (validación de existencia antes de eliminar, uso de `intval()` para el identificador). La respuesta 200 OK confirma que la operación se completó y que se eliminó exactamente una fila.

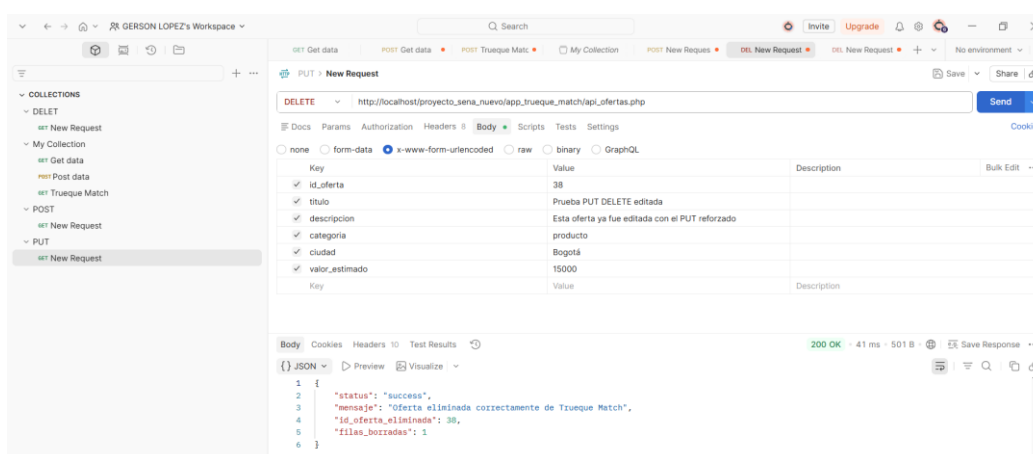


Figura 11. Prueba en Postman del método DELETE sobre `api_ofertas.php`, con respuesta exitosa.

14.3 Casos de error controlados

Además de probar los casos donde todo sale bien, verifiqué que el sistema responda de forma controlada ante datos inválidos: por ejemplo, al enviar una categoría de oferta con mayúscula en vez de minúscula, el sistema la rechaza con un mensaje de error claro en vez de guardar un dato

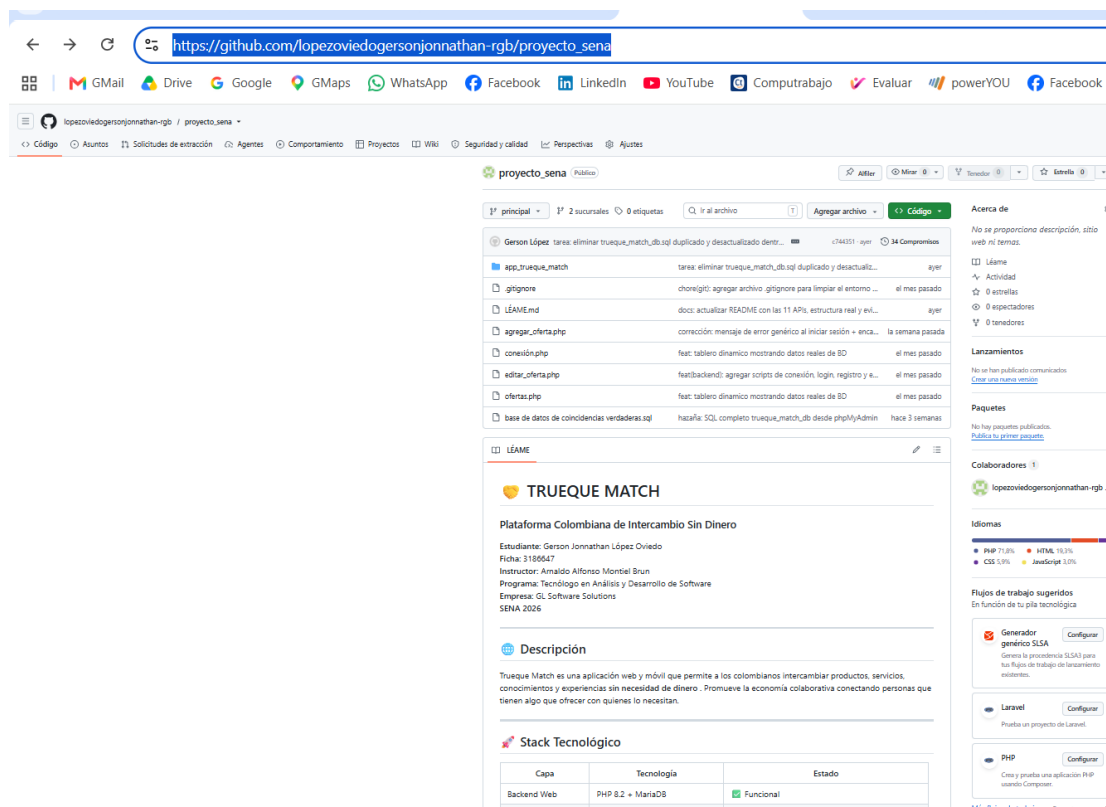
inconsistente en la base de datos. Esto confirma que las validaciones de cada API funcionan tanto para el camino exitoso como para el camino de error.

14.4 Resumen de pruebas por módulo

Módulo probado	Operaciones verificadas	Resultado
Autenticación	Login con credenciales válidas	Exitoso
Ofertas	Crear, editar, eliminar, listar	Exitoso
Solicitudes	Crear, aceptar, listar	Exitoso
Trueques	Proponer, completar (con actualización automática de ofertas)	Exitoso
Evaluaciones	Crear (con recálculo automático de reputación)	Exitoso
Favoritos	Agregar, listar, quitar	Exitoso
Notificaciones	Crear, marcar como leída	Exitoso
Reportes	Crear, resolver	Exitoso

15. Control de Versiones

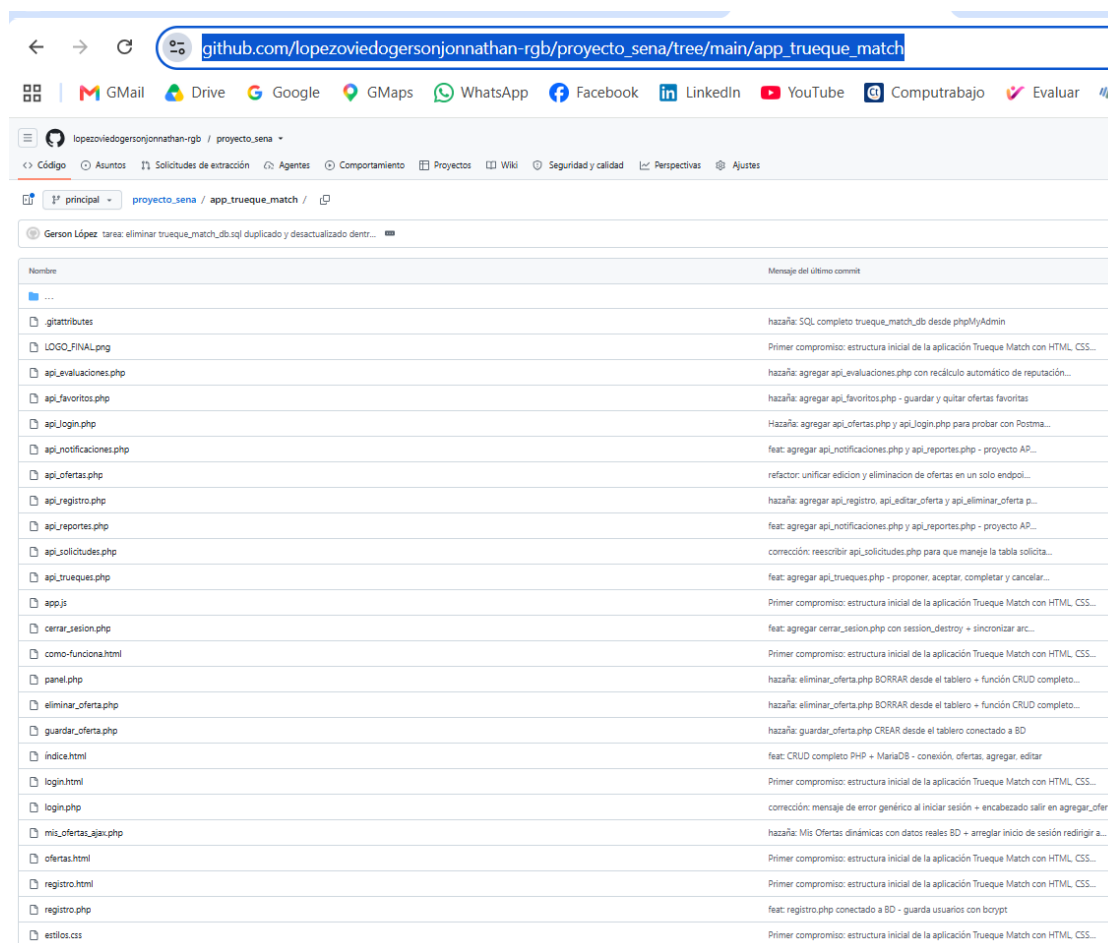
Todo el código del proyecto está versionado con Git y respaldado en un repositorio público de GitHub, lo que permite tener un historial completo de cada cambio realizado, con la fecha y una descripción de qué se modificó.



The screenshot shows the GitHub repository page for 'proyecto_sena' by Gerson López. The repository is public and has 2 commits. The README file is visible, describing 'TRUEQUE MATCH' as a Colombian platform for exchanging products and services without money. The README includes a 'Stack Tecnológico' table showing the use of PHP 8.2, MariaDB, and Laravel.

Capa	Tecnología	Estado
Backend Web	PHP 8.2 + MariaDB	Funcional

Figura 12. Repositorio de Trueque Match en GitHub, con más de 30 commits registrados.



Nombre	Mensaje del último commit
...	
gitattributes	hazña: SQL completo trueque_match_db desde phpMyAdmin
LOGO_FINAL.png	Primer compromiso: estructura inicial de la aplicación Trueque Match con HTML, CSS...
api_evaluaciones.php	hazña: agregar api_evaluaciones.php con recálculo automático de reputación...
api_favoritos.php	hazña: agregar api_favoritos.php - guardar y quitar ofertas favoritas
api_login.php	Hazña: agregar api_ofertas.php y api_login.php para probar con Postma...
api_notificaciones.php	feat: agregar api_notificaciones.php y api_reportes.php - proyecto AP...
api_ofertas.php	refactor: unificar edición y eliminación de ofertas en un solo endpoi...
api_registro.php	hazña: agregar api_registro, api_editar_oferta y api_eliminar_oferta p...
api_reportes.php	feat: agregar api_notificaciones.php y api_reportes.php - proyecto AP...
api_solicitudes.php	corrección: reescribir api_solicitudes.php para que maneje la tabla solicita...
api_trueques.php	feat: agregar api_trueques.php - proponer, aceptar, completar y cancelar...
app.js	Primer compromiso: estructura inicial de la aplicación Trueque Match con HTML, CSS...
cerrar_sesion.php	feat: agregar cerrar_sesion.php con session_destroy + sincronizar arc...
como-funciona.html	Primer compromiso: estructura inicial de la aplicación Trueque Match con HTML, CSS...
panel.php	hazña: eliminar_oferta.php BORRAR desde el tablero + función CRUD completo...
eliminar_oferta.php	hazña: eliminar_oferta.php BORRAR desde el tablero + función CRUD completo...
guardar_oferta.php	hazña: guardar_oferta.php CREAR desde el tablero conectado a BD
indice.html	feat: CRUD completo PHP + MariaDB + conexión, ofertas, agregar, editar
login.html	Primer compromiso: estructura inicial de la aplicación Trueque Match con HTML, CSS...
login.php	corrección: mensaje de error genérico al iniciar sesión + encabezado salir en agregar_oferta
mis_ofertas_aja.php	hazña: Mis Ofertas dinámicas con datos reales BD + arreglar inicio de sesión redirigir a...
ofertas.html	Primer compromiso: estructura inicial de la aplicación Trueque Match con HTML, CSS...
registro.html	Primer compromiso: estructura inicial de la aplicación Trueque Match con HTML, CSS...
registro.php	feat: registro.php conectado a BD - guarda usuarios con bcrypt
estilos.css	Primer compromiso: estructura inicial de la aplicación Trueque Match con HTML, CSS...

Figura 13. Listado de archivos dentro de la carpeta app_trueque_match en el repositorio remoto.

Cada commit sigue una convención de nombres que indica el tipo de cambio realizado: feat para una funcionalidad nueva, fix para una corrección, refactor para una mejora sobre código ya existente sin cambiar su comportamiento, docs para cambios en documentación, y chore para tareas de mantenimiento. Esta convención facilita revisar el historial y entender la evolución del proyecto sin tener que leer el código completo de cada versión.

16. Configuración de Servidores y Base de Datos

El ambiente de desarrollo y pruebas se configuró sobre XAMPP, una distribución que integra Apache (servidor web) y MariaDB (base de datos) de forma local, ideal para el desarrollo antes de un posible despliegue en un servidor de producción.

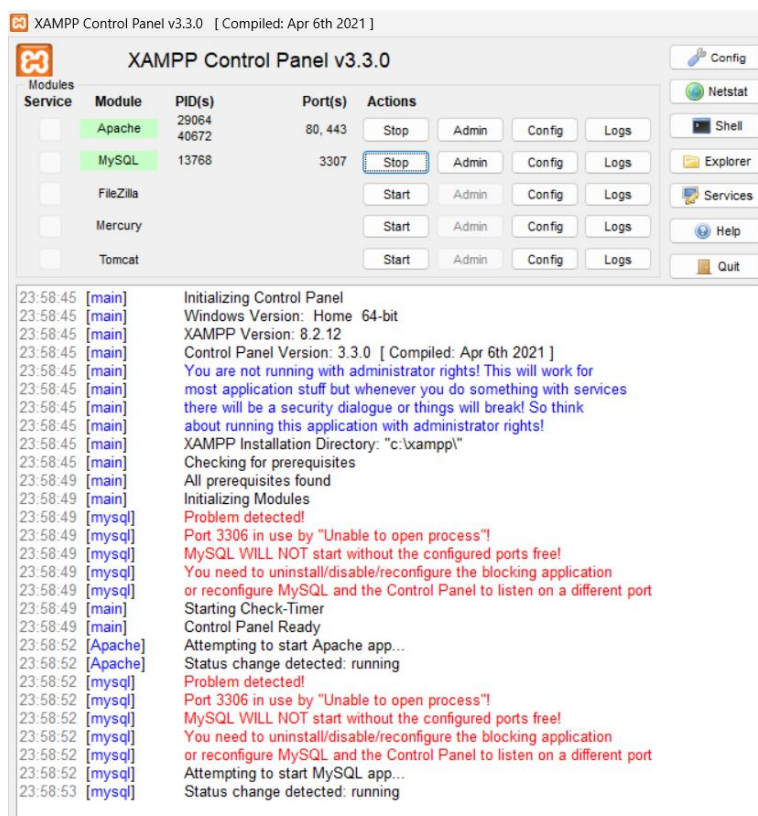


Figura 14. Panel de control de XAMPP con Apache y MySQL/MariaDB activos.

Un detalle importante de mi configuración es que **MariaDB** corre en el puerto 3307 en vez del puerto por defecto 3306, porque ese puerto ya estaba en uso por otro programa en mi computador. Esto lo tuve que reflejar explícitamente en el archivo **conexion.php** para que todas las consultas se dirigieran al puerto correcto.

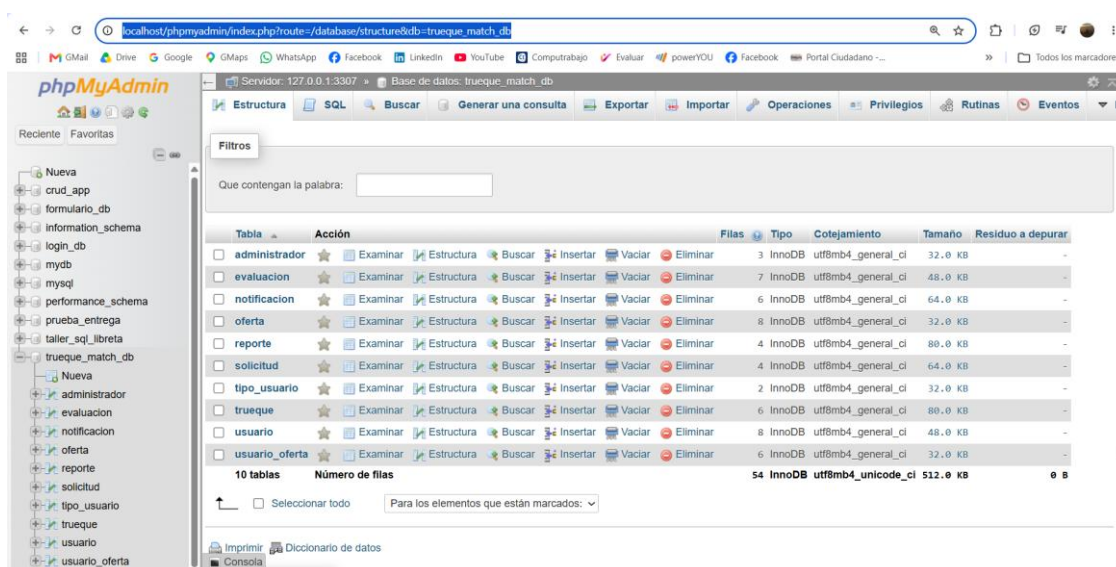


Figura 15. Estructura de la base de datos trueque_match_db en phpMyAdmin, con sus 10 tablas.

La base de datos *trueque_match_db* contiene 10 tablas normalizadas hasta la tercera forma normal (3FN), lo que evita la duplicación innecesaria de información y mantiene la integridad de los datos mediante llaves foráneas.

17. Ambientes de Desarrollo y Pruebas

Elemento	Configuración
Sistema operativo	Windows (Laptop Acer Core 7)
Servidor web	Apache 2.4 (incluido en XAMPP), puerto 80
Motor de base de datos	MariaDB, puerto 3307 (personalizado)
Lenguaje backend	PHP 8.2
Ruta del proyecto	C:\xampp\htdocs\proyecto_sena_nuevo\
URL de pruebas locales	http://localhost/proyecto_sena_nuevo/app_trueque_match/
Herramienta de pruebas de API	Postman (Desktop)
Repositorio remoto	GitHub - github.com/lopezoviedogersonjonnathan-rgb/proyecto_sena
Ambiente móvil	Expo Go sobre Android (dispositivo físico Redmi 9)

El ambiente de desarrollo y el ambiente de pruebas, en este proyecto, coinciden en la misma máquina local, lo cual es una práctica habitual en proyectos formativos de este tamaño. No existe todavía un ambiente de producción público, ya que el proyecto está en etapa de aprendizaje dentro del programa de formación SENA.

18. Conclusión

El desarrollo de Trueque Match a partir de la integración de sus módulos componentes demuestra la aplicación práctica de los conceptos de arquitectura por capas, modularidad, control de versiones y buenas prácticas de codificación estudiados durante la formación. Cada módulo del sistema, desde la autenticación hasta las 11 APIs REST, fue codificado siguiendo un patrón consistente, documentado con comentarios línea por línea, y verificado con pruebas reales antes de darlo por terminado.

El resultado es una plataforma funcional de principio a fin: un usuario puede registrarse, iniciar sesión, publicar una oferta, recibir una solicitud de intercambio, formalizar un trueque, calificar a la otra persona y ver cómo su reputación se actualiza automáticamente, todo respaldado por una base de datos normalizada y un repositorio de control de versiones con más de 30 commits documentados.

Quedan identificadas como mejoras futuras la implementación de una validación real de sesión en todas las APIs y la construcción de un panel de administración protegido, las cuales no bloquean la entrega actual pero enriquecerían la seguridad y la funcionalidad del sistema en una siguiente etapa del proyecto.